



Course Code: EEE373
Course Title: Embedded System Design

Prepared by:

Afrida Islam
ID: 21221030
Section: 02

Mid Assignment

Answer to the question no. 1

(a) The resistor connected between a microcontroller input pin and the positive voltage supply (V_{CC}) with a switch button, is known as a pull-up resistor. Pull-up resistors are necessary for inputting data into a microcontroller to ensure stable and consistent readings. When the input pin is not driven by any external device, the resistor ensures that the input pin reads a specific logic high level (1). This prevents the pin from floating, which can cause unpredictable behaviour due to noise or interference.

In order to connect internal pull-up resistors to an ATmega32, first we need to set the corresponding bit in the PORT register to activate the pull-up resistor and then we need to clear the corresponding bit in the Data Direction Register (DDR) to configure the pin as an input.

(b)

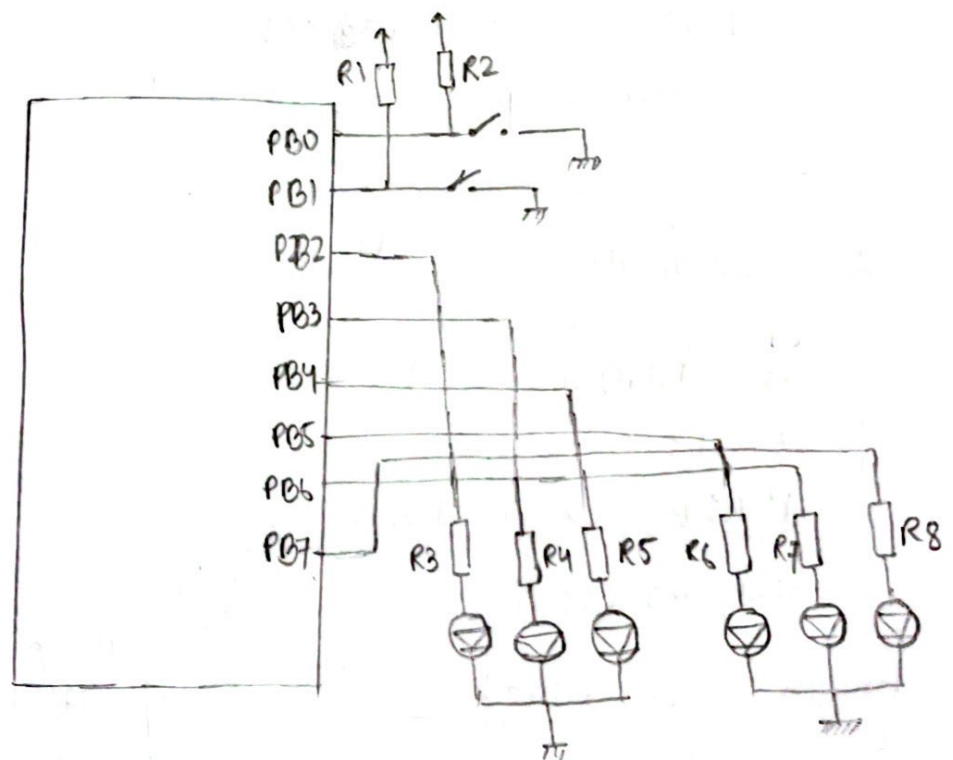


fig: Circuit diagram of the given system

```
(c) #include <mega32.h>
```

```
#include <delay.h>
```

```
void main (void)
```

```
{
```

```
    DD RB = 0b11111100; // PB0 and PB1 are inputs and PB2 to PB7 are  
                        LED outputs.
```

```
    while (1)
```

```
    {
```

```
        // Left turn switch connected to PB0
```

```
        if ( ( PINB & (1<< PB0 PINB0) ) == 0 ) // Active low on PB0
```

```
        {
```

```
            PORTB = 0b00000000; // Clear all LEDs
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB2); // Turn on PB2
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB3); // Turn on PB3
```

```
PORTB |= (1<< PINB4); //
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB4); // Turn on PB4
```

```
            delay_ms (100);
```

```
        }
```

```
        // Right turn switch connected to PB1
```

```
        else if ( ( PINB & (1<< PINB1) ) == 0 ) // Active low on PB1
```

```
        {
```

```
            PORTB = 0b00000000; // Clear all LEDs
```

```
            delay_ms (100);
```

```
PORTB |= (1<< PINB5); // Turn on PB5
```

```
delay_ms(100);
```

```
PORTB |= (1<< PINB6); // Turn on PB6
```

```
delay_ms(100);
```

```
PORTB |= (1<< PINB7); // Turn on PB7
```

```
delay_ms(100);
```

```
}
```

```
else
```

```
{
```

```
PORTB = 0b00000000; // Turn off all LEDs
```

```
}
```

```
}
```

Answer to the question no.2

(a) No, the ATmega 32 does not normally handle an interrupt within an interrupt. This is because the ATmega 32 automatically disables global interrupt when an interrupt service routine (ISR) is executed. The ATmega 32 disables the interrupt within an ISR is to ensure that the system does not encounter unexpected behaviour due to multiple ISRs trying to execute simultaneously. If multiple interrupts were allowed then a ~~more~~ higher priority interrupt may disrupt the current on going execution which further can cause data inconsistency.

During the execution of ISR, other interrupts can be allowed by manually enabling global interrupts within an ISR using the 'seiv' function. This 'seiv' function re-enables global interrupts, which further allows higher-priority or also lower-priority interrupts to occur.

(b)

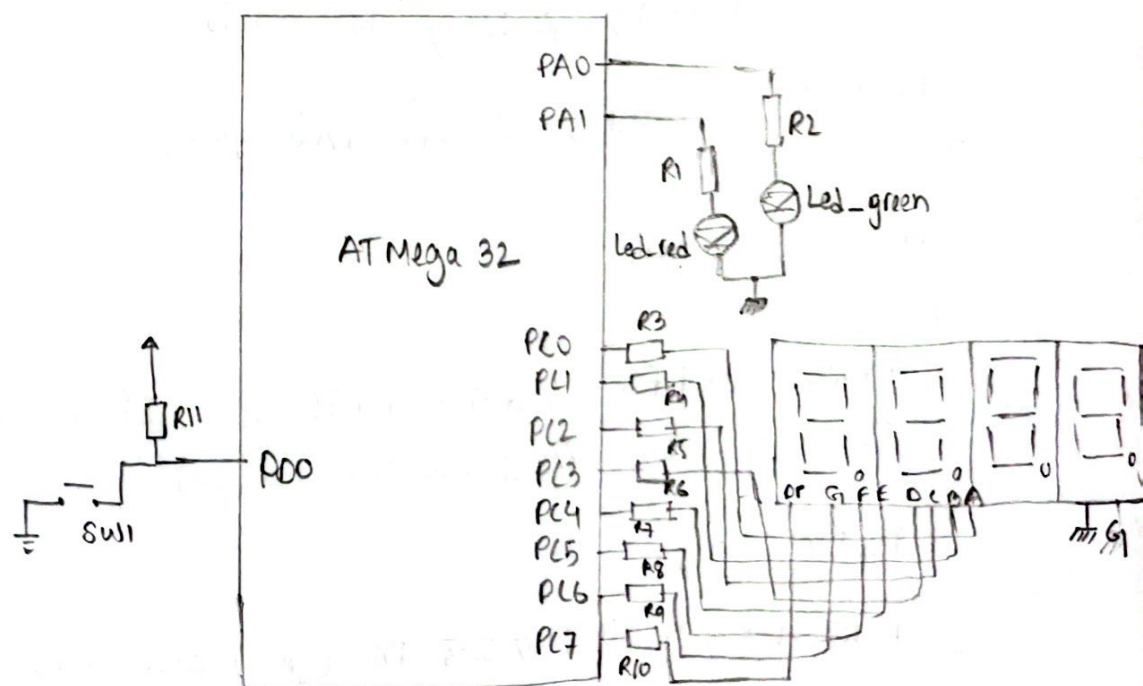


Fig: Circuit diagram of the given system

```
(c) #include <mega.h>
#include <delay.h>
```

```
int j;
```

```
// External interrupt INTO
```

```
: interrupt [EXT-INT0] void button(void) {
```

```
    unsigned char cathode[6] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6F};
```

```
    // 7 segment Hex codes for 0 to 5
```

```
    PORT A = 0x02; // sets PA1 (red) and clears PA0 (green)
```

```
    // Countdown on 7-segment display from 5 to 0
```

```
    for (j=5; j>0; j--)
```

```
    {
```

```
        PORT C = cathode[j] // Display countdown
```

```
        delay_ms(1000); // 1-second delay.
```

```
    }
```

```
    PORT C = 0; // clears 7-segment display
```

```
    delay_ms(10000); // delay 10 seconds
```

```
    PORT A = 0x01; // sets PA0 (green) and clears PA1 (red)
```

```
}
```

```
void main(void)
```

```
{
```

```
    DDRC = 0xFF; // define PORTC as output for 7-segment display
```

```
    DDRA = 0x03; // define PA0 and PA1 as outputs for LEDs
```

```
    PORT C = 0;
```

```
    PORT A = 0x01; // sets PA0 (green) and clears PA1 (red)
```

```
GICR |= (1 << INTO); // Enables external interrupt INTO
MCUCR |= (1 << ISC01); // Triggers interrupt on falling edge INTO

#asm ("sei") // Enables global interrupt

while (1)
{
    // Infinite loop waiting for interrupt
}
```